3008ICT Deep Learning

Assignment 1 - Programming

- **Due:** Friday 17th April 2026, 11:59 PM.
- **Total Marks:** 30%:

School of ICT, Griffith University
Trimester 1, 2026

Instructions:

- **Late submission:** Late submission is allowed but a penalty applies. The penalty is defined as the reduction of the mark allocated to the assessment item by 5% of the total weighted mark for the assessment item, for each working day that the item is late. A working day is defined as Monday to Friday. Assessment items submitted more than five working days after the due date will be awarded zero marks.

- **Extension:** You can request an extension of time on one of two grounds: medical or special circumstances (e.g. special family or personal circumstances, unavoidable commitments). Please note that workloads such as other assignments are not eligible grounds for extensions.

  All requests must be made through the myGriffith portal.

- **Individual Work:** You must complete this assignment **individually** and submit authentic work. This should be your own work. Anyone caught plagiarising will be penalised and reported to the university.

Objectives:

✓ Implement neural networks and learning algorithms.

✓ Evaluate the performance of simple learning systems with several tiny neural networks on a real-world dataset.

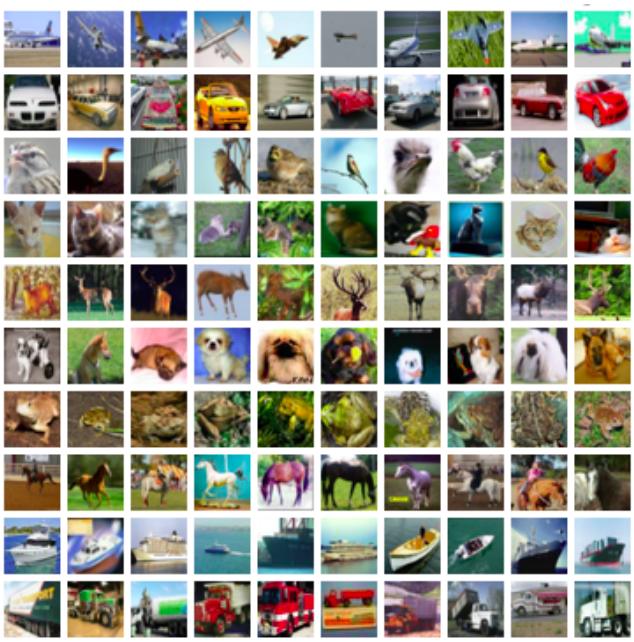
Your code should be written in **PyTorch** (Python)**.** In rare cases, students may be allowed to use another programming language, but approval by the course convenor is required. You are allowed to use some functions in torchvision but NOT torchvision.models or similar.

Classification using Neural Networks

In this task, you need to implement a widely adopted machine learning algorithm, Neural Networks, in PyTorch. You will train and test some neural networks in different depths with the datasets provided to you and experiment with various settings of hyperparameters.

Dataset Introduction

CIFAR-10 is a foundational machine learning benchmark. It consists of 60,000 32x32 color images containing one of 10 object classes, with 6000 images per class. It was collected by Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton. A sample of the dataset is given below.



Specifically, each sample is a RGB image, associated with a label from 10 classes. Each image is 32x32 pixels, namely, consists of 1024 pixels in total. Each pixel has three channels, and each channel is represented as an integer between 0 and 255. Each sample is assigned to one of the following ten classes:

0. airplane
1. automobile
2. bird
3. cat
4. deer
5. dog
6. frog
7. horse

8. ship
9. truck

Implementation requirements

Your task is to implement neural network learning algorithms that create verified network architectures:

1) Softmax regression.
2) Convolutional neural network (CNN) containing at **least two** Convolution layers, one Max Pooling layer, and one Linear layer with activation function ReLU.

Your code should be in PyTorch and as concise as possible. An example output is included in the appendix in the end of this document.

Required Experiments

You should conduct the following experiments and document the results. The best way is to demonstrate results in Tables and/or Figures e.g learning curves of training and testing. First, you need to split CIFAR-10 dataset into train set, validation set, and test set. **Use a fixed seed e.g. (0) to reproduce the same results every time your run the code**. If you can't run the experiments in your laptop, you can use Google Colab, watch this video for how to use Google Colab.

(a) **CNN Depth Analysis**

Train CNN models with different depths: 2, 8, 16, and 32 convolutional layers.

- Compare the training loss, validation loss, and accuracy for each model
- Does increasing the network depth improves the performance?
- Do deeper networks become harder to train? Provide explanation.
- Now incorporate techniques such as residual connection, dropout and weight decay to deeper models (16, 32) and analyse their effects on the network.
- Plot the training and validation curves for each experiment and explain the trends.

(b) **Learning Rate Analysis**

Select one specific network architecture for your CNN (e.g. best performance), train the network on the train dataset with different learning rates: 0.000001, 0.0001, 0.001, 0.01, 1.0.

- Compare the training error, validation error, and test error for each learning rate.
- Which learning rate achieves the highest test accuracy?
- Which learning rates lead to slow training?
- Explain how learning rate affects both model performance and training time.

(c) **Mini-batch Size Study**

Use the same network architecture as in (b), train the network on the train dataset with the same settings as in (a), but with different mini-batch sizes = 1, 8, 16, 64, 256.

- Which mini-batch size achieves the highest test accuracy?
- Which mini-batch size is the slowest in training time?

Report Requirements

You should write a report for the task, and the report should include an appendix for the source code. You might wish to include a few necessary screenshots in the report.

The file for the document must be in one of the two formats: Microsoft Word (.doc/.docx), or Adobe PDF (.pdf).

There is no page limit for the report. But basic criteria of scientific writing should be followed.

a. Software design: a detailed description of your code and how it works. Include details on key functions and data structures you are using in your program.
b. The test results and diagrams.
c. Discuss the results, include any observations or insights you gained from running tests.
d. Write a conclusion paragraph that summaries and explains your findings.

Submission

You should submit the following documents through the link provided in L@G by the due date:

1. A copy of your report as specified above. Name the file: *firstname_surname_Snumbe*r_A1_report (for example, Bart_Simpson_s123456_A1_report.pdf)
2. A Jupyter notebook (.ipynb file). Name the file: *firstname_surname_Snumbe*r_A1_code.ipynb

Note: Make sure your code is well documented and include comments to explain key parts.

Marking Scheme

A detailed marking rubric will provided with this document in L@G.

****** End of Assignment Description ******

Appendix. Example Output (in the next page):

```
============================================================
FASHION-MNIST NEURAL NETWORK TRAINING
============================================================
Model: LINEAR
Input size: 784
Output size: 10
Epochs: 20
Learning rate: 0.01
Batch size: 4
Data directory: ./data
Validation ratio: 0.2
Random seed: 42
============================================================
Loading Fashion-MNIST dataset...
Training data shape: (60000, 784), Labels shape: (60000,)
Test data shape: (10000, 784), Labels shape: (10000,)
Number of classes: 10

Creating validation split with 20% of training data...
  Class 0: 6000 -> Train: 4800, Val: 1200
  Class 1: 6000 -> Train: 4800, Val: 1200
  Class 2: 6000 -> Train: 4800, Val: 1200
  Class 3: 6000 -> Train: 4800, Val: 1200
  Class 4: 6000 -> Train: 4800, Val: 1200
  Class 5: 6000 -> Train: 4800, Val: 1200
  Class 6: 6000 -> Train: 4800, Val: 1200
  Class 7: 6000 -> Train: 4800, Val: 1200
  Class 8: 6000 -> Train: 4800, Val: 1200
  Class 9: 6000 -> Train: 4800, Val: 1200

Final split:
  Training set: 48000 samples
  Validation set: 12000 samples

Created Linear Classifier

Starting training for 20 epochs...
Learning rate: 0.01, Batch size: 4
Epoch   1/20 | Train Loss: 1.0567 | Val Loss: 1.0077 | Test Loss: 0.9906 | Train Acc: 83.00% | Val Acc: 82.42% | Test Acc: 83.00% | Time: 4.06s
         *** New best validation accuracy! ***
Epoch   5/20 | Train Loss: 0.7669 | Val Loss: 0.9023 | Test Loss: 0.8831 | Train Acc: 84.76% | Val Acc: 84.09% | Test Acc: 84.71% | Time: 4.10s
         *** New best validation accuracy! ***
Epoch  10/20 | Train Loss: 0.7298 | Val Loss: 0.8707 | Test Loss: 0.8530 | Train Acc: 85.66% | Val Acc: 84.56% | Test Acc: 85.34% | Time: 4.63s
         *** New best validation accuracy! ***
Epoch  15/20 | Train Loss: 0.7160 | Val Loss: 0.8668 | Test Loss: 0.8512 | Train Acc: 86.09% | Val Acc: 84.76% | Test Acc: 85.23% | Time: 4.18s
         *** New best validation accuracy! ***
Epoch  20/20 | Train Loss: 0.7063 | Val Loss: 0.8653 | Test Loss: 0.8502 | Train Acc: 86.21% | Val Acc: 84.86% | Test Acc: 85.44% | Time: 4.42s
```